

canvas-mcp

Servidor MCP local para Canvas LMS

Dossier técnico — Proyecto personal de código abierto
Julio de 2026

Repositorio: github.com/admin978/canvas-mcp · PyPI: [canvas-local-mcp](https://pypi.org/project/canvas-local-mcp/)

Índice

1. Introducción y motivación	3
2. Objetivos	3
3. Análisis	3
3.1. Requisitos funcionales	3
3.2. Requisitos no funcionales	4
4. Diseño y arquitectura	4
4.1. Decisiones de diseño	4
4.2. Estructura del proyecto	5
5. Implementación	5
5.1. Herramientas MCP expuestas	5
5.2. Paginación de la API	5
5.3. Configuración y credenciales	5
5.4. Volcado masivo (dump.py)	6
6. Pruebas y aseguramiento de la calidad	6
6.1. Estrategia de pruebas	6
6.2. Un error real encontrado por las pruebas	6
6.3. Análisis estático e integración continua	6
7. Distribución y despliegue	7
8. Seguridad y privacidad	7
9. Resultados y conclusiones	7
9.1. Posible interés para estudiantes y docencia	7
10. Trabajo futuro	8

1. Introducción y motivación

Canvas LMS es la plataforma de gestión del aprendizaje utilizada por miles de instituciones educativas. Su interfaz, sin embargo, está diseñada principalmente para el profesorado: el estudiante se encuentra con una experiencia fragmentada, sin búsqueda transversal entre cursos y con notificaciones que llegan tarde o no llegan.

Al mismo tiempo, el Model Context Protocol (MCP) se ha consolidado como estándar abierto para conectar asistentes de IA con servicios externos mediante «herramientas» que el modelo puede invocar. Este proyecto une ambas cosas: canvas-mcp es un servidor MCP que expone la API REST oficial de Canvas como herramientas, de forma que cualquier cliente compatible (Claude Code, Claude Desktop u otros) puede consultar cursos, tareas, calificaciones o anuncios mediante lenguaje natural.

El énfasis del diseño es «local-first»: el servidor se ejecuta en la máquina del usuario, el token de acceso nunca sale de ella y no existe ningún intermediario de terceros entre el cliente y la API oficial de Canvas.

Este documento describe un proyecto personal, desarrollado al margen de cualquier asignatura, ya publicado como software libre en PyPI y en el MCP Registry oficial. Se presenta al profesorado con un doble objetivo: dar a conocer una herramienta que puede resultar útil a otros estudiantes de la institución y recoger sugerencias de quien conoce Canvas desde el otro lado del aula.

2. Objetivos

- **Funcional:** Exponer las operaciones de lectura más útiles de la API de Canvas como herramientas MCP para el flujo de trabajo diario de un estudiante.
- **Privacidad:** Que el token de acceso personal permanezca siempre en la máquina del usuario, sin servicios intermedios.
- **Auditabilidad:** Código pequeño y auditable: cualquier usuario debe poder leer el servidor completo antes de confiarle su token.
- **Distribución:** Instalación en un solo comando desde PyPI y registro en el MCP Registry oficial para su descubrimiento.
- **Calidad:** Suite de pruebas automatizadas e integración continua que garanticen la calidad en cada cambio.

3. Análisis

3.1. Requisitos funcionales

- RF1 — Listar cursos activos del usuario autenticado.

- RF2 — Listar tareas de un curso con fecha de entrega, puntuación y estado de la entrega propia.
- RF3 — Consultar módulos, anuncios y páginas wiki de un curso.
- RF4 — Consultar calificaciones actuales, por curso o agregadas.
- RF5 — Consultar la agenda: planificador, próximos eventos y lista de pendientes.
- RF6 — Descarga masiva del material de los cursos para indexado sin conexión (herramienta auxiliar).

3.2. Requisitos no funcionales

- RNF1 — El token de Canvas se almacena localmente (`~/canvas.env`, permisos 600) y nunca atraviesa servicios de terceros.
- RNF2 — Transporte stdio: sin puertos abiertos ni superficie de red propia.
- RNF3 — Dependencias mínimas (httpx y el SDK oficial de MCP); la herramienta de volcado usa solo biblioteca estándar.
- RNF4 — Compatibilidad con Python 3.10 a 3.13, verificada en CI.
- RNF5 — Solo operaciones de lectura: el servidor no puede modificar nada en Canvas, lo que acota el riesgo.

4. Diseño y arquitectura

La arquitectura es deliberadamente mínima: un proceso local que traduce llamadas a herramientas MCP en peticiones HTTPS a la API oficial.

```
[cliente MCP] —stdio→ [server.py] —HTTPS→ [API de Canvas]
```

4.1. Decisiones de diseño

- **Transporte stdio:** El cliente lanza el servidor como subprocesso y se comunica por entrada/salida estándar. Es el transporte recomendado para servidores locales: no hay autenticación de red que gestionar porque no hay red.
- **Local-first:** Frente a soluciones SaaS que piden el token del estudiante, aquí el token vive en un fichero local con permisos restrictivos. La confianza se deposita únicamente en código que el usuario puede leer.
- **Un solo módulo auditable:** El servidor cabe en un único módulo de ~150 líneas. Es una decisión consciente frente a una arquitectura por capas: para 10 herramientas de lectura, la capa de indirección costaría más de lo que aporta y dificultaría la auditoría.
- **Dos clientes HTTP:** `server.py` usa `httpx` (cliente moderno, ya requerido por el SDK de MCP); `dump.py` usa `urllib` de la biblioteca estándar para que el

volcado funcione sin el entorno del servidor. La paginación de Canvas (cabecera Link, rel="next") se implementa en ambos.

- **Filtrado de respuesta:** Cada respuesta de la API se reduce a los campos útiles (id, nombre, fechas, puntuación...) antes de devolverla al modelo, reduciendo el consumo de contexto del LLM.

4.2. Estructura del proyecto

```
canvas_local_mcp/  
├─ server.py    # servidor MCP: 10 herramientas de lectura  
├─ dump.py     # volcado masivo de material (stdlib)  
└─ __init__.py # versión del paquete  
tests/         # 21 pruebas con API simulada  
.github/workflows/ # CI y publicación en MCP Registry
```

5. Implementación

5.1. Herramientas MCP expuestas

Herramienta	Función
list_courses	Cursos matriculados (id, nombre, código)
list_assignments	Tareas de un curso, con fecha límite, puntos y estado de entrega
list_modules	Módulos del curso con sus elementos
list_announcements	Anuncios del curso
get_page	Página wiki de un curso por su slug
get_file_info	Metadatos de un fichero, incluida la URL de descarga
get_grades	Calificaciones actuales, por curso o globales
planner_items	Elementos del planificador en un rango de fechas
upcoming_events	Próximos eventos (≈ 2 semanas)
todo	Lista de tareas pendientes del usuario

Cada herramienta se declara con el decorador `@mcp.tool()` de FastMCP (SDK oficial de Python); la firma tipada y el docstring generan automáticamente el esquema JSON que ve el modelo.

5.2. Paginación de la API

Canvas pagina todas las colecciones mediante la cabecera HTTP Link. La función interna `_get()` sigue los enlaces rel="next" acumulando resultados hasta agotar las páginas, con `per_page=100` para minimizar viajes.

5.3. Configuración y credenciales

El primer arranque lee `~/.canvas.env` (`CANVAS_BASE_URL` y `CANVAS_TOKEN`). Si faltan variables, el servidor termina con un mensaje que indica exactamente cómo crearlas. Las variables de entorno del proceso tienen prioridad sobre el fichero, lo

que permite inyectar secretos desde un gestor externo (se incluye un envoltorio para Bitwarden Secrets Manager en scripts/dev.sh).

5.4. Volcado masivo (dump.py)

La herramienta canvas-local-mcp-dump descarga el material de todos los cursos activos (o de los indicados por id): ficheros a través de los módulos, páginas, enunciados de tareas y anuncios, generando una estructura de carpetas navegable y JSON con los metadatos. Los nombres de fichero se sanean con una expresión regular que conserva caracteres del español (á, ñ...).

6. Pruebas y aseguramiento de la calidad

6.1. Estrategia de pruebas

La suite (pytest, 21 casos) prueba el servidor contra una API de Canvas simulada mediante httpx.MockTransport, sin tocar la red ni necesitar credenciales: cada prueba define un manejador que devuelve respuestas JSON controladas y verifica la petición generada (ruta, parámetros) y la transformación de la respuesta.

- Carga de configuración: lectura de ~/.canvas.env, prioridad de variables de entorno y error claro si faltan credenciales.
- Cliente HTTP: seguimiento de la paginación Link, objetos únicos, propagación de errores HTTP.
- Herramientas: forma exacta de la salida de list_courses, list_assignments (con y sin entregas) y get_grades (ambas ramas).
- Contratos: las 10 herramientas registradas con descripción, coherencia entre server.json, la versión del paquete y .canvas.env.example.
- dump.py: saneado de nombres, paginación, y semántica de descarga (omitir existentes, crear directorios).

6.2. Un error real encontrado por las pruebas

Al escribir la prueba de paginación afloró un defecto real: al seguir el enlace rel="next", el código pasaba params={} a httpx, que interpreta el diccionario vacío como «sustituir la query» y eliminaba ?page=2 de la URL. El resultado era volver a pedir la página 1 indefinidamente: un bucle infinito en cualquier colección con más de una página. La corrección (pasar None en las peticiones de continuación) quedó cubierta por la propia prueba de regresión. Es un ejemplo de manual del valor de probar contra dobles de la API: el fallo solo se manifestaba con colecciones grandes y habría sido difícil de diagnosticar en producción.

6.3. Análisis estático e integración continua

El estilo se verifica con ruff (reglas de pycodestyle, pyflakes, isort, pyupgrade y bugbear). Un workflow de GitHub Actions ejecuta lint y pruebas en cada push y pull request sobre una matriz de Python 3.10, 3.11, 3.12 y 3.13.

7. Distribución y despliegue

- **PyPI:** Publicado como `canvas-local-mcp`; instalación con `pip install canvas-local-mcp`, que registra los comandos `canvas-local-mcp` y `canvas-local-mcp-dump`.
- **MCP Registry:** El fichero `server.json` describe el servidor según el esquema oficial y un workflow de GitHub Actions lo publica en el registro público de MCP en cada etiqueta de versión, autenticándose sin secretos mediante OIDC de GitHub.
- **Versionado:** Versionado semántico (actualmente 0.1.1, estado alfa); una prueba automática impide que la versión del paquete y la del registro diverjan.

8. Seguridad y privacidad

- El token de acceso personal se guarda en `~/.canvas.env` con permisos 600 y solo se envía a la instancia oficial de Canvas de la institución, por HTTPS.
- No hay intermediarios: a diferencia de integraciones SaaS, ningún tercero ve las credenciales ni los datos académicos.
- Superficie mínima: transporte stdio (sin puertos), solo lectura (la API de escritura de Canvas no está expuesta) y ~300 líneas auditables en total.
- El repositorio no contiene secretos: se distribuye una plantilla (`.canvas.env.example`) y la publicación en registros usa OIDC en lugar de tokens almacenados.

9. Resultados y conclusiones

El proyecto cumple los objetivos planteados: cualquier estudiante puede instalar el paquete desde PyPI, registrar el servidor en su cliente MCP en un comando y preguntar en lenguaje natural «¿qué entregas tengo esta semana?» o «¿cómo voy en esta asignatura?», obteniendo datos en vivo de Canvas sin ceder su token a terceros.

Más allá del código, el proyecto ha recorrido el ciclo completo de un producto de software real: análisis del problema, diseño con compromisos justificados, implementación, una suite de pruebas que encontró y previno un defecto grave, integración continua multiversión, empaquetado, publicación en dos registros públicos y documentación de usuario.

9.1. Posible interés para estudiantes y docencia

- Cualquier alumno con un cliente MCP puede instalarlo hoy (`pip install canvas-local-mcp`) y usarlo como asistente personal sobre sus propios cursos, sin que la institución tenga que desplegar nada.

- La herramienta de volcado permite a un estudiante llevarse el material de sus cursos para estudiar sin conexión o indexarlo localmente.
- Como el proyecto completo son ~300 líneas de Python con pruebas y CI, puede servir de caso de estudio compacto en asignaturas de ingeniería del software: API REST reales, paginación, empaquetado, pruebas con dobles y publicación automatizada.
- El código es libre (MIT): se aceptan sugerencias, incidencias y contribuciones de profesores y alumnos.

10. Trabajo futuro

- Búsqueda transversal sobre el material descargado (indexado del volcado para consultas semánticas).
- Caché local con invalidación para reducir latencia y llamadas a la API.
- Herramientas de escritura opcionales y acotadas (por ejemplo, marcar elementos del planificador como completados).
- Notificaciones proactivas: avisar de nuevas calificaciones o anuncios sin que el usuario pregunte.
- Pruebas de integración opcionales contra una instancia real de Canvas (entorno de demostración de Instructure).